

Tools for Continuously Evaluating Distributed-System Qualities

James H. Hill, *Indiana University-Purdue University Indianapolis*

Douglas C. Schmidt, James R. Edmondson, and Aniruddha S. Gokhale,
Vanderbilt University

Model-driven engineering can enable agile development of distributed systems and continuous system integration to improve quality-of-service validation throughout the software life cycle.

Developers are increasingly using service-oriented middleware to develop distributed systems.¹ This middleware raises the abstraction level for software so that distributed-system developers can focus more on application-level concerns (for instance, business logic) rather than wrestle with infrastructure-level concerns (such as software adaptation, context-awareness, and life-cycle management). Service-oriented middleware also promotes reuse of business logic and services across heterogeneous application domains, thus facilitating the development of larger, more complex systems.²

As service-oriented distributed systems grow in size and complexity, ensuring that they conform to their specifications throughout the software life cycle becomes more difficult. This difficulty stems in part from the problem of *serialized-phasing development*,³ in which application-level entities are developed after infrastructure-level entities. Serialized-phasing development makes it difficult to evaluate end-to-end functional and quality-of-service (QoS) aspects until late in the software life cycle—for example, at system integration time.

Agile techniques help address functional aspects of serialized-phasing development by validating software functionality throughout the entire software life cycle.^{4,5} For example, test-driven development and continuous integration are agile techniques that validate functional quality by ensuring that software behaves as expected. The benefit of using agile techniques to improve QoS assurance of service-oriented distributed systems,

however, has not been demonstrated. Developers, therefore, need new techniques to help alleviate the complexity of serialized-phasing development and enable evaluation of QoS concerns throughout the software life cycle.

Model-driven engineering (MDE) is a promising solution for improving software development of service-oriented distributed systems.⁶ MDE techniques, such as domain-specific modeling languages (DSMLs),⁷ provide developers with visual representations of abstractions that capture key domain semantics and constraints. DSMLs also provide tools that transform models into concrete artifacts, such as source code or configuration files, that are tedious and error-prone to create manually using third-generation languages. Moreover, such artifacts often are not available early enough in the software life cycle to allow a proper evaluation of end-to-end QoS properties.

In this article, we discuss how we use DSMLs to realize agile techniques for continuously eval-

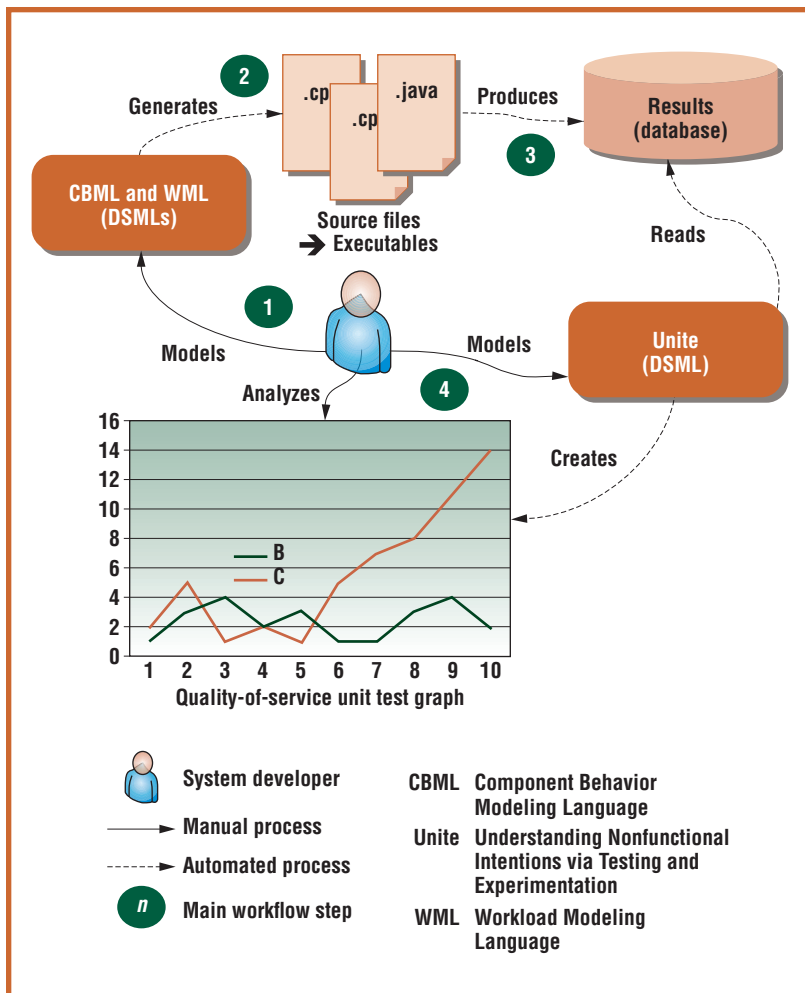


Figure 1. CUTS (Component Workload Emulator [Coworker] Utilization Test Suite) workflow and domain-specific modeling languages (DSMLs). Developers model behavior and workload using CBML and WML, respectively; generate a complete system for testing via emulation; and use Unite to analyze system performance.

uating service-oriented distributed-system QoS throughout the software life cycle.

The Agile QoS Assurance Process

Our approach is based on system execution modeling,⁸ which let developers conduct the following agile QoS assurance process:

1. Rapidly model the behavior and workload of the distributed system being developed, independently of its programming language or target environment (including the underlying networks, operating systems, and middleware platforms).
2. Synthesize a customized test system from models, including representative source code for the behavior and workload models, as well as the project and workspace files necessary to build the test system in its target environment.
3. Execute the synthesized test system on a representative target environment testbed to produce realistic empirical results at scale.
4. Analyze the test system's QoS in the context of

domain-specific constraints—such as scalability or end-to-end response time of synthesized test applications—to identify performance antipatterns,⁸ which are common system design mistakes that degrade QoS.

We've realized such agile techniques in an open source tool called CUTS (Component Workload Emulator [Coworker] Utilization Test Suite, www.cs.iupui.edu/CUTS). CUTS is a programming-language-, operating-system-, and middleware-independent DSML-based system execution modeling tool for service-oriented distributed systems.

We've used CUTS as a case study to qualitatively and quantitatively evaluate how DSML-based system execution modeling tools can support lightweight and adaptable software development and QoS assurance processes. Developers can use these tools to quickly pinpoint performance bottlenecks and other QoS concerns throughout the life cycle, rather than wrestle with low-level test implementations written in third-generation languages.

Ensuring QoS in Service-Oriented Distributed Systems

The following CUTS DSMLs provide the key capabilities necessary to ensure QoS in service-oriented distributed systems:

- The Component Behavior Modeling Language (CBML) DSML models component behavior.
- The Workload Modeling Language (WML) DSML models component workload.
- The Understanding Nonfunctional Intentions via Testing and Experimentation (Unite) DSML specifies QoS unit tests for performance analysis in distributed systems.

Figure 1 shows how the CUTS DSMLs map to the agile QoS assurance process described in the previous section. To focus the discussion, we describe how the QoS-Enabled Dissemination (QED) project (see the “Overview of the QED Project” sidebar) applies these CUTS DSMLs and their agile techniques.

Capability 1: Capturing Behavior and Workload

Service-oriented distributed systems are typically reactive and respond to inputs, such as events or remote method invocations. Such systems' behavior and workload are analogous to a sequence of actions that cause side effects, such as changing the value of a variable, querying a database, or

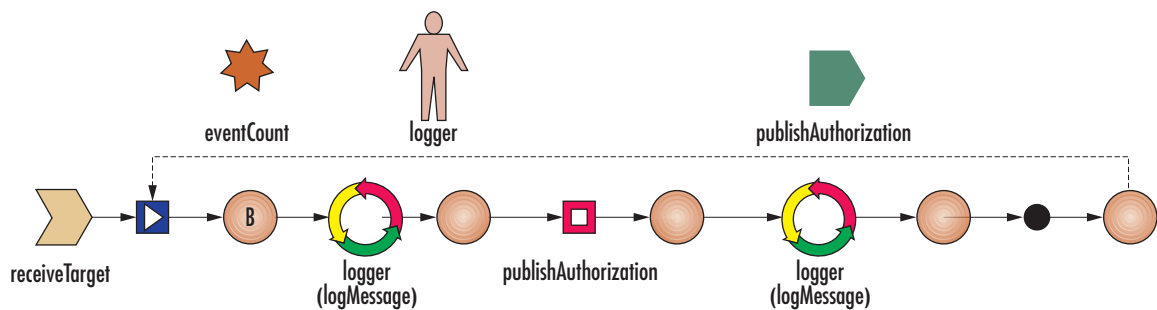


Figure 2. Behavior and workload model for a multistage workflow application component. This figure shows the behavior and workload, as an action-to-state sequence, that occurs when the component receives an event on the receiveTarget event source.

generating output events. DSML-based system execution modeling tools, therefore, must capture these properties to provide a lightweight adaptive process that rapidly models behavior and workload. These tools must also use intuitive domain-specific abstractions to avoid manually implementing distributed systems using tedious and error-prone third-generation programming languages.

We've realized this capability in CUTS by providing CBML and WML DSMLs (step 1 in Figure 1) to simplify modeling behavior and workload. Developers can use CBML to define the behavior of a *component*, which is software that encapsulates common services for individual entities of a service-oriented distributed system using action-to-state sequence diagrams. Likewise, developers use WML to characterize component workload by parameterizing actions in CBML—for example, by setting their values.

In CBML, *actions* represent operations, such as an application component receiving and sending an event or querying a database. *States* represent the current value of the system's variables. Each action can also cause an *effect*, which can cause a component's state to change (such as incrementing a component's event counter variable). In WML, *workload generators* (workers) represent *objects* (such as C++ or Java classes) that perform predefined behavior. *Worker actions* represent object methods—that is, the predefined behavior.

Figure 2 shows a partial behavior and workload model for a multistage workflow application component used to evaluate QED's QoS. In this figure, the behavior and workload of the receiveTarget event begins with an input action

Overview of the QED Project

The QoS-Enabled Dissemination (QED) project¹ (where QoS means quality of service) is a large-scale, multiteam collaborative project that focuses on QoS-enabled service-oriented infrastructure and applications in the Global Information Grid (GIG).² The GIG middleware must provide dependable and timely communication to applications and user scenarios that operate within dynamically changing conditions and environments, such as wireless ad hoc networks or bandwidth and resource-constrained situations. The QED middleware enhancements for the GIG, therefore, provide timely delivery of information to users in mobile scenarios, tailoring and prioritizing information according to mission needs and importance, and operating in a manner that's robust to failures and intermittent communications.

References

1. J. Loyall et al., "QoS Enabled Dissemination of Managed Information Objects in a Publish-Subscribe-Query Information Broker," Proc. SPIE Conf. Defense Transformation and Net-Centric Systems, vol. 7350, SPIE, 2009.
2. J.G. Grimes, Department of Defense Global Information Grid Architectural Vision: Vision for a Net-Centric, Service-Oriented DoD Enterprise, v. 1.0, US Dept. of Defense, 2007; <http://cio-nii.defense.gov/docs/GIGArchVision.pdf>.

(the leftmost square). After the initial input action, a sequence of actions (such as logMessage) and states define the behavior. The component has a worker named logger that reports user-defined information about the component.

Because QED developers model behavior and workload using DSMLs, they can easily adapt their models to test different scenarios, such as changes in QED's specification, or to evaluate different QoS concerns. For example, if QED developers must add new behavior and workload to log more or less information about the system, updating existing CBML and WML models is straightforward. Moreover, DSMLs shield developers from wrestling with low-level infrastructure details, which can be time-consuming and error-prone.

Capability 2: Generating Realistic Data

Ensuring QoS of service-oriented distributed systems requires continuously generating realistic data

```

1  receiveTarget (TargetEvent ev) {
2      {
3          // effect: update the eventCount
4          + ++eventCount;
5
6          (eventCount % publishRate == 0) {
7              // generate log message
8              . logger_ . logMessage (LM_INFO
9                  instanceName + ": Event " + eventCount +
10                     ": Received a TargetMio at " +
11                     System . currentTimeMillis ());
12
13             // create new event for publishing
14             JbiEvent <AuthorizationType> ev_1_ =
15                 JbiEvent <AuthorizationType> (
16                     AuthorizationType . );
17
18             ev_1_ . setPayload (1024);
19             ev_1_ . setMetadata (/* metadata */);
20
21             // publish event to GIG
22             . publishAuthorization_ .
23                 publishData (ev_1_);
24
25             // generate log message
26             . logger_ . logMessage (/* message */);
27         }
28     }
29     (Exception e) {
30         e . printStackTrace ();
31     }
32 }

```

Figure 3. Portion of automatically generated source code for a multistage workflow application component. This code corresponds to a portion of the behavior/workload model shown in Figure 2 and reflects a real portion of the source code for the component it emulates.

and results, such as components' service times, throughout the software life cycle. DSML-based system execution modeling tools, therefore, should provide this information. We've realized this capability in CUTS by leveraging model interpreters to automatically generate faux application components on the basis of the constructed behavior and workload models. These components conform to the interfaces of the actual components being developed. This conformance enables the incremental replacement of faux components as development evolves, thereby enabling continuous system integration and evaluation of QoS throughout the software life cycle.

After QED developers use CBML and WML to model the behavior and workload of components in the multistage workflow application (step 1 in Figure 1), they can use CUTS model interpreters to generate source code customized to their target environment (step 2 in Figure 1). Figure 3 shows a portion of the Java source code for the multistage workflow model in Figure 2.

By using DSML-based system execution modeling tools and model interpreters, QED developers can rapidly implement a complete test system and evaluate QoS. Moreover, because CBML and WML are independent of the target programming language, operating system, and middleware, existing models can scale and adapt to different environments. For example, the multistage workflow application uses the CUTS interpreter for the Java-based QED middleware. If QED developers want to generate source code for different environments (such as Microsoft's .NET Framework or OMG's Data Distribution Service), their existing models can remain unchanged, and the CUTS model interpreter will adapt as needed. This typically involves defining mapping functions from CBML and WML to the target architecture, and manually implementing them in the CUTS model interpreter.

Capability 3: Collecting and Analyzing Distributed-System Data

Collecting and analyzing QoS metrics in service-oriented distributed systems is difficult because the data often changes over time. System structure can also change over the life cycle—for example, by altering a component's interface, increasing the number of replicated components, or modifying system deployments by adding or removing connections between components—which can affect how data is analyzed. Data collection and analysis techniques, therefore, must adapt to the volatility in service-oriented distributed systems.

We've realized this capability in CUTS via the Unite DSML to analyze end-to-end QoS independently of data and system complexity, using system execution traces generated during a test run (step 4 in Figure 1). Developers can use Unite to define QoS unit tests, which include the following:

- *Log formats* identify the data to extract from system execution traces (such as the event count in the log message from the logMessage action in Figure 2, and line 8 in Figure 3). These log formats let Unite adapt to many variants of the same log format when extracting data for QoS metrics.
- *Causal relations* specify the order of occurrence for each log format in the system execution trace. These causal relations let Unite correlate and evaluate distributed data regardless of system structure and complexity.
- A user-defined *evaluation function* f (such as the end-to-end response time of a multistage workflow application based on variables in log formats for extracting data) lets Unite adapt

evaluation of complex QoS metrics without prior knowledge of data complexity and system complexity.

Developers can also define an *aggregation function*, such as SUM(f) and AVG(f), which combines multiple occurrences of a result, and a *grouping criteria*, which partitions results into sets before aggregation. By removing the aggregation function, developers can view the QoS unit test's data trend throughout the system's execution in its target environment. Figure 3 highlights two different log messages (lines 8 and 26) generated by a multistage workflow application component. Likewise, Figure 4 shows the corresponding Unite specification for locating the log messages generated in Figure 3. This specification helps evaluate QED QoS. The following simple equation calculates average service time on the basis of the partial specification in Figure 3.

$$f = \text{AVG}(LF_2.\text{sendTime} - LF_1.\text{recvTime})$$

The Unite DSML provides QED developers with a lightweight technique to ensure QoS by automatically extracting metrics of interest from system traces. Moreover, Unite doesn't require QED developers to understand distributed-system composition when analyzing extracted data. For example, if QED developers increase the number of components in the multistage workflow application or want to extract more or less data from system traces, Unite can adapt to such scenarios.

Evaluating Agile Techniques for QoS Assurance

We conducted experiments to evaluate the QoS of the QED and Global Information Grid (GIG) middleware by applying the CUTS DSMLs to a multistage workflow application that runs atop GIG middleware. The multistage workflow application consists of six different component stages, each represented by a rectangular object in Figure 5. The lines connecting each component represent communication channels passing through the QED middleware and GIG infrastructure. Each application component contains a CUTS behavior and workload model that stresses different parts of the QED and GIG middleware by increasing network traffic at controlled intervals. Each component also contains actions that log metrics for Unite to analyze the QED and GIG middleware performance.

We ran the experiments in a representative target environment testbed at the Institute for Soft-

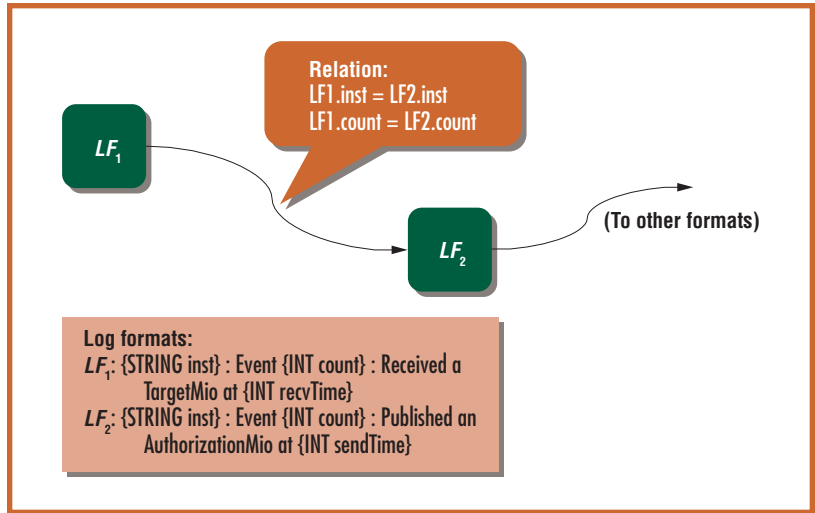


Figure 4. Partial Unite specification for a multistage workflow application QoS unit test. This specification defined by developers will be used by Unite to calculate the average service time of components on the basis of metrics extracted from system execution traces.

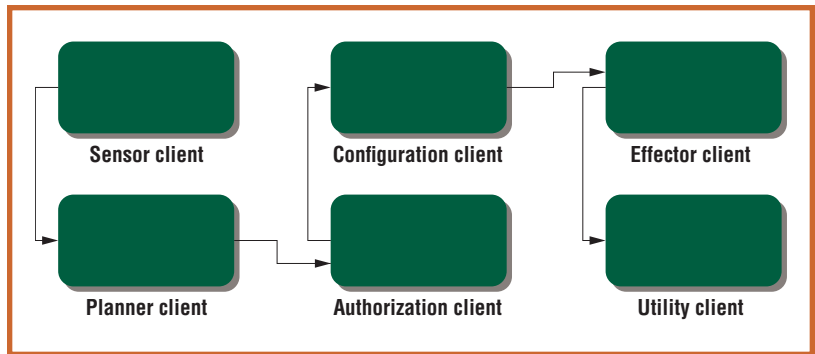


Figure 5. Structural model of the multistage workflow application. Each rectangular object represents a different component stage. The lines connecting each component represent communication channels passing through the QED middleware and GIG infrastructure.

ware Integrated Systems' (ISIS) ISISlab (www.isis-lab.vanderbilt.edu).

Baselining the GIG Middleware

The response time of the GIG middleware—that is, one-way latency for sending and receiving events between two components—is important because it helps determine the existing QoS capabilities of the GIG middleware and where the QED middleware can improve QoS relative to the GIG middleware baseline. James Hill and colleagues conducted experiments that measured the response time of events published by components in the multistage workflow application using the baseline GIG middleware (that is, before integrating QED capabilities).⁹

QED developers were also concerned about scalability because it affects the GIG middleware's data capacity, while also ensuring timely and reliable communication. This, therefore, is another area where QED can improve QoS relative to the baseline GIG middleware. So, we conducted

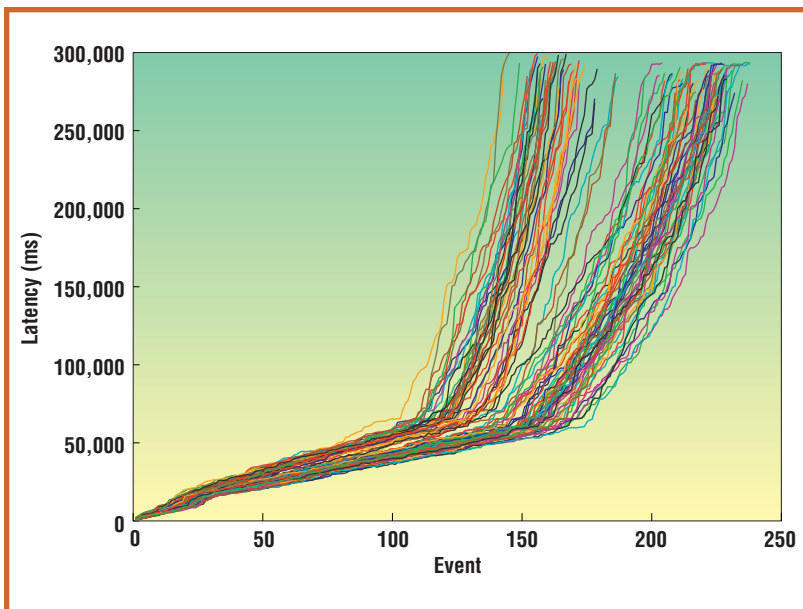


Figure 6. Evaluating the scalability of the Global Information Grid (GIG) middleware during early development stages. This figure was created by analyzing system execution traces using Unite. It shows that the GIG middleware doesn't handle events of higher priority differently than events of lower priority because they have the same performance outcome.

Table 1

CUTS* model elements vs. source lines of code for automatically generated code (all numbers are approximations)

Application	Model elements	Source lines of code
Client-server	20	530
Multistage workflow	80	1,760

*Component Workload Emulator (Coworker) Utilization Test Suite

scalability tests on the baseline GIG middleware using a client-server application model that contains fewer components than the multistage workflow application in Figure 5.

Figure 6 presents the results of executing a scalability test of the GIG middleware, consisting of 24 subscribers and 12 publishers receiving and sending two different event types at different priorities, respectively. To distribute application workload, we deployed the subscribers and publishers across three different hosts. As shown in this figure, the baseline GIG middleware doesn't differentiate service on the basis of event or workflow priority. Hence, this test confirmed the QED developer's new hypothesis based on the earlier results of Hill and his colleagues,⁹ and located another area of QoS concern that QED should address.

Evaluating the CUTS DSMLs

Applying agile techniques in DSML-based system execution tools helps reduce the effort of testing service-oriented distributed-system QoS. For example, the QED developers used the CUTS DSMLs to model the multistage workflow application's behavior and workload, and used the CUTS model interpreters to automatically generate complete test systems. Table 1 compares the number of elements against the automatically generated source lines of code (SLOC) for the different test systems that the QED developers used to evaluate the QoS of the GIG middleware. This table highlights that the number of modeling elements needed to define the test systems for the experiments conducted to baseline the GIG middleware is substantially less than the SLOC. Thus, the QED developers required less time and effort to generate and run tests in their target environment than to implement it manually.

The CUTS agile techniques also alleviate the complexity of analyzing results for the distributed systems using Unite. Table 2 quantifies the effort needed to analyze results for the client-server and multistage-workflow applications used to evaluate the GIG middleware. This table shows that the QoS unit test specification is a lightweight process, because the number of log formats and group specifications needed to process log messages that contain metrics of interest is substantially lower than the number of log messages collected during a test run that contain metrics of interest for evaluating the QoS unit test. Likewise, as system complexity grows, the number of log formats for the QoS unit test remains constant. Assuming log formats remain stable, Unite's QoS unit test specification process is thus a one-time effort for software developers.

On the basis of our results and experience in developing and applying CUTS to the QED and GIG middleware and their applications, we learned four lessons. First, DSML-based system execution modeling tools provide practical solutions that attack the problem of serialized-phasing development. As systems grow in size and complexity, alleviating the effects of serialized phasing becomes essential. Our experience in applying CUTS to the QED project shows that DSML-based system execution modeling tools can locate performance bottlenecks during early stages of the software life cycle and readily adapt to different scenarios with little effort.

Second, DSML-based system execution model-

ing tools are most effective on large-scale projects with relatively long software life cycles (one to two years or more). Also, learning the DSMLs doesn't require significant effort, the modeling process is manual, and managing this process requires a dedicated set of team members. As DSML-based system execution modeling tools become more automated, applying them to projects with shorter software life cycles will be easier and won't require dedicated team members to manage their processes.

Third, DSML-based system execution modeling tools are more suitable for system integrators than system developers because it allows system developers to focus on writing source code, and allows system integrators to focus on validating QoS while the system is being developed. Moreover, the effort needed to train system integrators on the DSMLs has greater payoff in the long run, especially because the models are independent of technology, programming language, and architecture, and thus can be reused across different application domains.

Fourth, DSML-based system execution modeling tools offer a cost-effective approach for validating QoS properties. System integrators are usually responsible for validating QoS properties throughout the software life cycle. Instead of requiring system integrators to become knowledgeable about the target architecture, DSMLs let them remain within their domain knowledge. Moreover, the DSMLs require less effort to realize integration tests, which helps shift system integrators' efforts to more important tasks, such as exploring the configuration space and its effects on QoS properties. ☞

Acknowledgments

CUTS and its associated DSMLs are available for download in open source format at www.cs.iupui.edu/CUTS.

References

1. M. Pezzini and Y.V. Natis, *Trends in Platform Middleware: Disruption Is in Sight*, tech. report, Gartner, 24 Sept. 2007; www.gartner.com/DisplayDocument?doc_cd=152076.
2. M. Bell, *Service-Oriented Modeling: Service Analysis, Design, and Architecture*, Wiley & Sons, 2008.
3. H.W.J. Rittel and M.M. Webber, "Dilemmas in a General Theory of Planning," *Policy Sciences*, vol. 4, no. 2, 1973, pp. 155–169.
4. P. Abrahamsson et al., "New Directions on Agile Methods: A Comparative Analysis," *Proc. 25th Int'l Conf. Software Eng. (ICSE 03)*, IEEE CS Press, 2003, pp. 244–254.
5. D. Saff and M.D. Ernst, "An Experimental Evaluation of Continuous Testing during Development," *Proc. ACM SIGSOFT Int'l Symp. Software Testing and Analysis*, ACM Press, 2004, pp. 76–85.
6. D.C. Schmidt, "Guest Editor's Introduction: Model-Driven Engineering," *Computer*, vol. 39, no. 2, 2006, pp. 25–31.

Table 2

Quality-of-service unit test specification vs. analysis in the Unite* DSML

Application	Formats:groups	Messages
Client-server	2:1	~54,660
Multistage workflow	2:1	~236,230

*Understanding Nonfunctional Intentions via Testing and Experimentation

About the Authors



James H. Hill is an assistant professor in the Department of Computer and Information Science at Indiana University-Purdue University Indianapolis. He is also the project lead for the CUTS (Component Workload Emulator [Coworker] Utilization Test Suite) system execution modeling tool. His research focuses on techniques for continuously validating quality-of-service properties on target architectures throughout the entire software life cycle. He has a PhD in computer science from Vanderbilt University. He is a member of IEEE and the ACM. Contact him at hillj@cs.iupui.edu.

Douglas C. Schmidt is a professor of computer science at Vanderbilt University. His research interests include patterns, optimization techniques, and empirical analyses of software frameworks and domain-specific modeling environments that facilitate the development of distributed real-time and embedded middleware and applications. He has a PhD in computer science from the University of California, Irvine. He is a member of IEEE. Contact him at d.schmidt@vanderbilt.edu.



James R. Edmondson is a research assistant in the Vanderbilt Institute for Software Integrated Systems at Vanderbilt University, where he is pursuing a PhD in computer science. His research interests include software testing methods, distributed systems, parallel programming, compilers, and algorithmic design. He is a member of IEEE and the ACM. Contact him at james.r.edmondson@vanderbilt.edu.

Aniruddha S. Gokhale is an assistant professor in the Department of Electrical Engineering and Computer Science at Vanderbilt University. His research focuses on investigating novel model-driven engineering solutions for system problems pertaining to distributed real-time and embedded systems. He has a DSc in computer science from Washington University in St. Louis, Missouri. He's a member of IEEE and the ACM. Contact him at a.gokhale@vanderbilt.edu.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.